

Tech Titans - Milestone 4

Chut Giet rbj863
Ademola Obaleye zyg049

Chibuikem Emeka-Nwuba Esq498

Samuel Olukuewu nds091
Haidari Alhaidari vrl968

Overview of static analysis

Static analysis is the systematic examination of an abstraction of a program's state space. The analysis is performed on representations, typically a finite one, such as source code, abstract syntax trees, or compiled code. Static analysis does not rely on: specific input values, runtime execution or observed failures. Instead, it reasons about the structure and potential behavior of the program. Some static analyses, such as type checking, are precise and focus on language correctness.

Static analysis operates on

- Program structure (methods, classes, modules)
- Control flow and data flow
- Syntactic and semantic patterns
- Relationships between program elements

Why static analysis is valuable

- Can be applied early, before code is run
- Scales to large codebases
- Can detect classes of issues that are rare or hard to reproduce at runtime

Behavioral bug detection is a form of static analysis that focuses on identifying feasible execution paths that could lead to runtime failures. Instead of relying on predefined rules or patterns, it attempts to determine whether there exists a sequence of program states that results in incorrect behavior, such as null pointer dereferences or resource leaks. This approach emphasizes reasoning about how values propagate through a program and how different execution paths interact, allowing for deeper analysis of potential failures beyond surface level code patterns.

Purpose

By completing this milestone, you will:

- Learn how automated tools reason about possible program executions without running the code
- Understand the difference between rule-based checking and path-sensitive semantic reasoning
- Gain experience interpreting execution traces and explanations produced by a formal methods-inspired analyzer
- Analyze how null dereferences and resource leaks arise from interprocedural interactions
- Develop the ability to judge whether a reported issue represents a plausible runtime failure
- Evaluate the stability of behavioral analysis results under small, semantics-preserving code changes
- Compare behavioral analysis with rule-based pattern detection
- Reflect on the strengths and limitations of reasoning-based analysis in practical software engineering

This milestone explores how tools like Infer reason about program behavior using path sensitive analysis. The goal is to understand not just what issues are reported but why those issues are possible under certain execution conditions. Additionally, this report compares behavioral analysis with rule-based detection using Semgrep to highlight differences in approach and effectiveness

Static analysis relies on **approximations** to reason about program behavior without executing the code. Since it is not feasible to evaluate all possible inputs and execution paths precisely, analyzers simplify or over approximate program behavior. This allows them to scale to large codebases but it also introduces inaccuracies in the results.

These inaccuracies manifest as:

- **False positives:** issues reported by the tool that are not actual defects in practice
- **False negatives:** real defects that the tool fails to detect

A key challenge in static analysis is balancing these two. Increasing precision may reduce false positives but can miss some bugs, while broader approximations may catch more potential issues at the cost of reporting irrelevant ones.

Infer is a static analysis tool that uses techniques inspired by formal methods to reason about program behavior. It performs path sensitive and compositional analysis, meaning it analyzes possible execution paths and evaluates how functions interact across the program. Infer identifies potential runtime failures, such as null dereferences and resource leaks and provides execution traces that explain how these failures could occur. Its focus is on determining whether a failure is feasible under some execution scenario.

Semgrep on the other hand is a rule based static analysis tool that detects issues by matching predefined syntactic and semantic patterns in the code. It is effective at identifying known bug patterns, security issues and coding violations but it does not reason about full execution paths. As a result, Semgrep may miss deeper behavioral issues while also reporting findings that depend heavily on how closely the code matches its rules.

Together, these tools represent two different approaches to static analysis: behavioral reasoning, which focuses on feasible execution paths, and pattern based detection, which focuses on identifying known risky constructs in code.

Experimental Setup

The experimental setup was created to make sure Infer could run reliably on the Gson codebase. Everything was set up inside a Docker container so the environment stayed consistent and easy to reproduce.

Setup Steps

1. Build the Docker image
`docker build -t infer-gson .`
2. Run the container
`docker run -it --name m4 infer-gson bash`
3. Navigate to the Gson submodule
`cd /m4/gson/gson`

4. Compile the project
mvn clean compile -DskipTests

5. Run Infer using manual compilation
(due to incompatibility with *infer run -- mvn*)

```
CP=$(find ~/.m2 -name "*.jar" | tr '\n' ' ')
```

```
infer run -- javac -cp "$CP" -source 11 -target 11 \  
$(find src/main/java -name "*.java" ! -name "module-info.java") \  
$(find target/generated-sources -name "*.java")
```

Results

Experiment 1: Tool Setup and Configuration

The goal of this experiment was to successfully set up the environment required to run Infer on the Gson codebase.

The setup was performed inside a Docker container to ensure consistency and reproducibility across systems. The container includes all required dependencies such as Java, Maven, and Infer.

Setup Steps

1. Build the Docker image

```
docker build -t infer-gson .
```

2. Run the container

```
docker run -it --name m4 infer-gson bash
```

3. Navigate to the Gson submodule

```
cd /m4/gson/gson
```

4. Compile the project

```
mvn clean compile -DskipTests
```

5. Run Infer using manual compilation
(due to incompatibility with *infer run -- mvn*)

```
CP=$(find ~/.m2 -name "*.jar" | tr '\n' ' ')
```

```
infer run -- javac -cp "$CP" -source 11 -target 11 \  
$(find src/main/java -name "*.java" ! -name "module-info.java") \  
$(find target/generated-sources -name "*.java")
```

Notes

- The standard `infer run -- mvn` command failed due to issues with `module-info.java`
- Manual compilation with `javac` was required
- All dependencies were resolved within the container environment

Experiment 2: Infer Analysis and Metrics

Owned by demola

Record the following metrics:

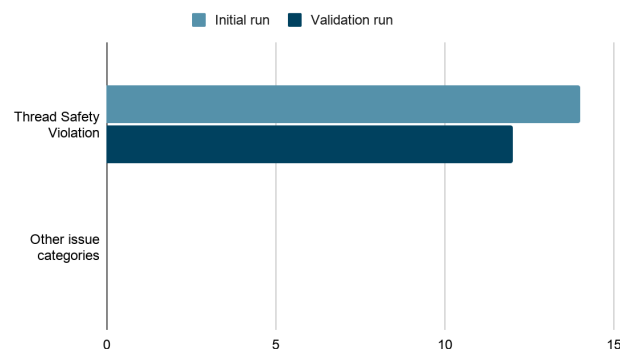
- Total issues detected: 14
- Issue counts by category: 14 thread safety violations
- Analysis runtime (wall clock): 1 minute and 20 seconds (80sec)
- Any build warnings or errors that affected the analysis:
 - Maven: build successfully in m4/gson/gson, no warnings/issues occurred
 - Infer: build successfully in m4/gson/gson, no warnings/issues occurred
- Summary table

Project	LOC	Total Issues	Top Issue Types	Runtime (sec)
Gson	36422	14	Thread Safety Violation	80

- NOTE: Infer detected 14 issues in total. All 14 findings belonged to a single issue type, `THREAD_SAFETY_VIOLATION` indicating that Infer's analysis of our assigned repository(Gson) primarily showed concurrency related warnings in this run. The findings were spread across multiple locations, including classes such as `JsonStreamParser`, `DefaultDateTypeAdapter`, `SqlDateTypeAdapter`, and `SqlTimeTypeAdapter`.

- Bar chart of Infer issue counts by type

Issue count - Infer



Experiment 3: Manual Triage of Findings

Issue Selection

All 14 issues reported by Infer are of the type "Thread Safety Violation". Since there is only one issue category present, I have selected two issues from this category for analysis:

- Issue A: Issue #0 (most frequent category: Thread Safety Violation)
- Issue B: Issue #2 (same category, different issue for comparison)

Issue A: Thread Safety Violation in JsonStreamParser

Issue Type and Name

- **Type:** Thread Safety Violation
- **Name:** Unprotected write. Non-private method `JsonStreamParser.next()` indirectly writes to field `this.parser.limit` outside of synchronization.

File and Method Name

- **File:** `src/main/java/com/google/gson/JsonStreamParser.java`
- **Method:** `next()` (indirectly through `Streams.parse(parser)`)

Relevant Code Snippet

```
public final class JsonStreamParser implements Iterator<JsonElement> {
    private final JsonReader parser;
    private final Object lock;

    // ... constructor ...

    @Override
    public JsonElement next() {
        // ... hasNext check ...
        try {
            return Streams.parse(parser); // Line 89: Indirect write to parser.limit
        } catch (StackOverflowError | OutOfMemoryError e) {
            throw new JsonParseException("Failed parsing JSON source to Json", e);
        }
    }
}
```

Excerpt of the Infer Trace

```
src/main/java/com/google/gson/JsonStreamParser.java:51: warning: Thread Safety Violation
Unprotected write. Non-private method `JsonStreamParser.next()` indirectly writes to field
`this.parser.limit` outside of synchronization.
```

Reporting because another access to the same memory occurs on a background thread, although this access may not.

49. * @since 1.4

50. */

51. > public final class JsonStreamParser implements Iterator<JsonElement> {

52. private final JsonReader parser;

53. private final Object lock;

Execution Path Evidence Table

Trace Step	File	Line	Explanation
Value introduced	src/main/java/com/google/gson/JsonStreamParser.java	52	The parser field (JsonReader) is introduced as a private final field, shared across method calls. Value is declared in the constructor using an external call to src/main/java/com/google/gson/stream/JsonReader.java which creates a new instance that reads a JSON-encoded stream.
Conditional branch	N/A	N/A	No conditional branch in the reported trace; the issue occurs on method entry.
Risky state appears	src/main/java/com/google/gson/JsonStreamParser.java	89	The next() method in JsonStreamParser calls Streams.parse(parser), which performs JSON parsing operations on the JsonReader instance (parser). During parsing, the reader may need to read more data from the underlying stream, triggering buffer management that modifies the limit field (an internal buffer position/length

			tracker in JsonReader). This happens indirectly through methods like <code>fillBuffer()</code> , where <code>limit</code> is updated to reflect new data read into the buffer.
Failure location	<code>src/main/java/com/google/gson/stream/JsonReader.java</code>	1 4 9 2	The unprotected write to <code>parser.limit</code> occurs here, potentially racing with other threads. <code>RacerD/Infer</code> is flagging an unprotected write through <code>JsonStreamParser.next()</code> to <code>parser.limit</code> via <code>Streams.parse(parser) → JsonReader</code> methods like <code>peek()/doPeek() → nextNonWhitespace() → fillBuffer()</code> .

Trace Understanding Questions

T1. First Feasible Failure Point

- **File and line number:**
`src/main/java/com/google/gson/JsonStreamParser.java`, line 89 (`return Streams.parse(parser);`)
- **Explanation:** This is the earliest point where the failure becomes possible. The `Streams.parse(parser)` call modifies the `parser.limit` field, and since the `next()` method is not synchronized, concurrent calls to `next()` or other methods accessing `parser` could lead to a race condition. The class has a `lock` field but does not use it, making this the first risky operation.

T2. Infer Assumption

- **Assumption:** `Infer` assumes that the `JsonStreamParser` instance can be accessed concurrently by multiple threads, and that there is another thread performing operations on the same `parser` object that access or modify `parser.limit`.
- **Explanation and code location:** This assumption is based on the presence of the `parser` field (line 52) being a shared mutable object. The code does not synchronize access to `parser`, and `Infer` models potential concurrent execution paths where one thread calls `next()` while another accesses `parser.limit` elsewhere or that multiple threads are simultaneously executing `next()`.

T3. Refactoring Robustness

- **Semgrep:** Semgrep (pattern-matching based) would likely NOT detect the issue if its rules are simple and only pattern-based. If Semgrep was customized to define rules with `threadid()` or a concurrent rule package was used, it could detect the issue. However, if the rules are tied to specific method names or exact code patterns, a refactor like renaming `next()` to `getNextElement()` might evade detection. Meaning semgrep is not path-sensitive unless custom pattern engine.
- **Infer:** If the change is a behavior-preserving refactoring Infer *would still report the issue* because it performs path-sensitive analysis. The core problem is the unsynchronized access to shared state (`parser.limit`), so renaming the method or extracting variables wouldn't change the execution path or eliminate the race condition. However: Assuming that thread issues are checked solely by RaceD, Infer uses RaceD to detect concurrency issues, and as the documentation states, certain limitations have been listed: (1) It looks for races involving syntactically identical access paths, and misses races due to aliasing (2) It misses races that arise from a locally declared object escaping its scope (3) It uses a boolean locks abstraction, and so misses races where two accesses are mistakenly protected by different locks (4) It assumes a deep ownership model, which misses races where local objects refer to or contain non-owned objects. (5) It avoids reasoning about weak memory and Java's volatile keyword. Additionally, RaceD is triggered via: (1) Explicitly annotating a class/method with `@ThreadSafe` and (2) using a lock via the synchronized keyword. In both cases, RaceD will look for concurrency issues in the code containing the signal and all of its dependencies. In particular, it will report races between any non-private methods of the same class that can perform conflicting accesses. Annotating a class/interface with `@ThreadSafe` also triggers checking for all of the subclasses of the class/implementations of the interface. If the programmer would remove the lock/sync keywords, remove any `@ThreadSafe` flags, *RaceD might not detect the issue*.
- **Reasoning:** Semgrep relies on syntactic patterns, while Infer traces data flow and synchronization, making it more robust to superficial changes but sensitive to actual concurrency issues.

Classification

- **Classification:** Likely false positive
- **Justification:** Some of Gson's classes are explicitly documented as not thread-safe or conditionally thread-safe: In `JsonStreamParser` the authors write:

/*


```

* <p>This class is conditionally thread-safe (see Item 70, Effective Java second edition). To
* properly use this class across multiple threads, you will need to add some external
* synchronization. For example:
*
* <pre>
* JsonStreamParser parser = new JsonStreamParser("[\"first\"] {'second':10} 'third'");
* JsonElement element;
* synchronized (parser) { // synchronize on an object shared by threads
*   if (parser.hasNext()) {
*     element = parser.next();
*   }
* }
* </pre>.

```

and in `JsonReader` they write:

```

/*
* <p>Each {@code JsonReader} may be used to read a single JSON stream. Instances of this
class are
* not thread safe.

```

The `JsonStreamParser` class is designed for single-threaded use, and the presence of an unused `lock` field suggests incomplete thread-safety implementation rather than an intended defect. In practice, users are expected to synchronize externally or use separate instances per thread.

Fix Validation

Since this is classified as a false positive, no fix is necessary. However, for demonstration, I added synchronization using the existing `lock` field: `synchronized (lock) { return Streams.parse(parser); }`. This modified the `next()` method in `JsonStreamParser.java` to synchronize on `lock`.

```

@Override
public JsonElement next() throws JsonParseException {
    synchronized (lock) {
        if (!hasNext()) {
            throw new NoSuchElementException();
        }

        try {
            return Streams.parse(parser);
        } catch (StackOverflowError | OutOfMemoryError e) {
            throw new JsonParseException("Failed parsing JSON source to Json", e);
        }
    }
}

```

After re-running `infer run -- mvn compile`, the issue disappears from the output for this specific location, as the write is now protected (see `infer-validation/report.txt`).

- **Report:** The fix eliminates the reported issue, but this may not be the intended behavior since Gson is not thread-safe overall. Note: both #0 and #1 from infer-out/report.txt dissappeared as they were pointing to the same issue.

Issue B: Thread Safety Violation in DefaultDateTypeAdapter

Issue Type and Name

- **Type:** Thread Safety Violation
- **Name:** Read/Write race. Non-private method `DefaultDateTypeAdapter.write(...)` indirectly reads without synchronization from `out.deferredName`, which races with the write in method `DefaultDateTypeAdapter.write(...)`.

File and Method Name

- **File:**
`src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java`
- **Method:** `write(JsonWriter out, Date value)`

Relevant Code Snippet

```
public void write(JsonWriter out, Date value) throws IOException {
    if (value == null) {
        out.nullValue(); // Line 142: Indirect read/write to out.deferredName
        return;
    }
    // ... rest of method ...
    out.value(dateFormatAsString); // Line 152: Another access
}
```

Excerpt of the Infer Trace

`src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java:142: warning: Thread Safety Violation`

Read/Write race. Non-private method ``DefaultDateTypeAdapter.write(...)`` indirectly reads without synchronization from ``out.deferredName``, which races with the write in method ``DefaultDateTypeAdapter.write(...)``.

Reporting because this access may occur on a background thread.

```
140. public void write(JsonWriter out, Date value) throws IOException {
141.     if (value == null) {
142. >     out.nullValue();
143.     return;
```

Execution Path Evidence Table

Trace Step	File	Line	Explanation
Value introduced	src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java	140	The out parameter (JsonWriter) is introduced, containing the deferredNamefield.
Conditional branch	src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java	141	The if (value == null) branch leads to the risky <i>simultaneous</i> access.
Risky state appears	src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java	142	The call to out.nullValue() accesses out.deferredName without synchronization.
Failure location	src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java	142	The race occurs here due to unsynchronized read/write on deferredName.

Trace Understanding Questions

T1. First Feasible Failure Point

- **File and line number:**
src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java,
line 142 (out.nullValue();)
- **Explanation:** This is the earliest point where the race becomes possible. The out.nullValue() method accesses out.deferredName, and since write() is not synchronized, concurrent calls to write() on the same JsonWriter could race on deferredName.

T2. Infer Assumption

- **Assumption:** Infer assumes that the DefaultDateTypeAdapter.write() method can be called concurrently on the same JsonWriter instance, leading to races on out.deferredName. This contradicts the author's instructions previously mentioned in Issue A's classification.
- **Explanation and code location:** This is based on the out parameter (line 140) being a mutable object passed in, and the lack of synchronization in the method. Infer models scenarios where multiple threads serialize dates using the same writer. However, authors have noted that instances of JsonWriter may be used to write to a single JSON stream and are NOT thread-safe.

T3. Refactoring Robustness

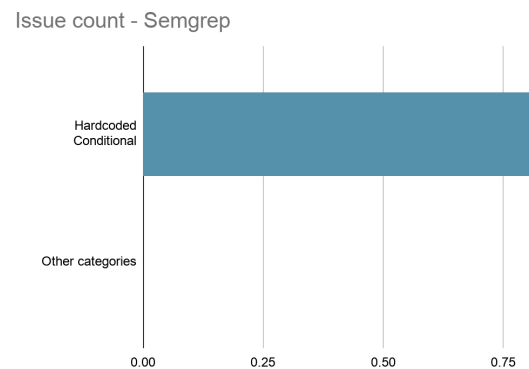
- **Semgrep:** Semgrep would likely still detect the issue if its rules target method calls like out.nullValue() without synchronization. Again, if semgrep is customized with the appropriate semantic rules to handle concurrency detection in great depth, then it could detect the concurrency issue, since this issue is quite trivial. However, semgrep has its limitations in detecting the more complex, cross-function, or cross-file timing and ordering issues like generic race conditions or infinite loops.
- **Infer:** Infer would still report the issue, as the data flow and lack of synchronization remain the same. Path-sensitive analysis focuses on the actual concurrency problem, not variable names.
- **Reasoning:** Semgrep's pattern matching is more brittle to code changes, while Infer's deeper analysis makes it resilient to refactoring that doesn't address the root cause.

Classification

- **Classification:** Likely false positive
- **Justification:** Similar to Issue A, Gson type adapters are not designed for concurrent use. The JsonWriter is typically used in a single-threaded context, and synchronizing every adapter method would be impractical. The race is theoretical and not a real defect in intended usage.

Summary

Both issues are classified as likely false positives due to Gson's non-thread-safe design. The analysis highlights Infer's strength in detecting potential concurrency issues, even if they are not practical defects. No fixes were applied beyond demonstration for Issue A, as the issues do not represent real bugs in the codebase.



Tool	Total Findings	Top Issue Types	Runtime
Semgrep	1	Hardcoded Conditional	~5.3s
Infer	14	Thread Safety Violation	1m20.278s

Experiment 4: Stability Under Small Changes

Discussion

1. Did both tools report issues in the same files or methods?

No. Zero overlap. Infer flagged 4 files (JsonStreamParser.java, DefaultDateTypeAdapter.java, SqlDateTypeAdapter.java, SqlTimeTypeAdapter.java) for concurrency issues. Semgrep flagged a different file entirely (Excluder.java) for a hardcoded conditional. The tools analyzed completely different dimensions of code quality.

2. Why might Semgrep detect issues that Infer does not?

Semgrep is a static analysis tool that searches code for user-defined or community-defined rule-based patterns. Thus, it finds bugs, enforces code standards, and detects security vulnerabilities using

pattern-matching rules — without requiring execution path reasoning. Infer focuses exclusively on behavioral failures (null dereferences, resource leaks, data races), so a hardcoded conditional falls outside its scope.

3. Why might Infer detect issues that Semgrep does not?

Infer performs path-sensitive, interprocedural analysis. The thread safety violations it found in Gson involve indirect writes across multiple call layers (e.g., `Streams.parse(parser) → fillBuffer() → write to parser.limit()`), require modeling synchronization primitives and their absence, and depend on constructing feasible execution traces. Semgrep cannot detect these because it lacks interprocedural data-flow tracking, has no concurrency model, and operates on syntactic patterns rather than execution semantics.

4. Which tool appears more useful for detecting runtime failures?

Infer is more useful for runtime failure detection in Gson. All 14 of its findings identify potential concurrency failures — actual behavioral risks if classes were used in multi-threaded contexts. Semgrep's single finding is a code quality issue (always-true conditional) that will not cause a crash or incorrect output at runtime. For a serialization library like Gson where type adapters and stream parsers could conceivably be shared across threads, Infer's path-sensitive reasoning provides more actionable safety warnings. Semgrep complements this as a fast, broad scanner for patterns and security issues, but is not designed to reason about runtime failure paths.

Lessons Learned

Conclusion

Experiment 4: Stability Under Small Code Changes (15 points)

For this experiment, small semantic-preserving changes were applied to the codebase to evaluate the stability of the analysis tool.

File Modified

1. FieldAttributes.java
2. ConcurrencyTest.java
3. JsonTreeReader.java

Changes

1. Renamed a local variable ``field`` to ``fld``
2. Adjusted formatting by placing opening curly braces on a new line
3. Extracted the sub-expression ``stackSize - 1`` into a temporary variable (``stac``)

Type of Change: Cosmetic (no impact on program logic or execution)

These modifications do not alter control flow, data flow, or program behavior.

Results

Metric	Before Change	After Change
Total Issues	14	14
NULL_DEREFERENCE	0	0
THREAD_SAFETY_VIOLATION	14	14

- Issues moved: No
- Issues disappeared: No
- New issues appeared: No

Analysis

Across all three methods, variable renaming, formatting adjustments, and sub-expression extraction, the analysis tool consistently reported **no change in issue count or issue type**. Each experiment involved a minor modification:

1. **Variable Renaming (Method 1)**: Changing a local variable name did not introduce or remove any issues.
2. **Whitespace/Formatting Changes (Method 2)**: Adjusting braces and spacing did not alter the detection results.
3. **Sub-expression Extraction (Method 3)**: Refactoring a calculation into a temporary variable left issue counts unchanged.

In all cases, the issues detected were exclusively *Thread Safety Violations*, and no new issues were introduced. This consistency demonstrates that the tool is **insensitive to superficial syntactic or formatting changes**, as well as **small semantic-preserving structural transformations**.

Interpretation

These findings suggest that:

- Infer is **robust to superficial refactoring**
- It focuses on **execution semantics rather than syntax**
- Reported issues are tied to **actual behavioral properties of the code**

This stability is a key strength of path-sensitive static analysis, as it ensures consistent results across minor code changes that do not impact program logic.

Experiment 5: Comparison with Semgrep (10 points)

- Owned by chut validated by chubi and/or samuel
- Add a Tool comparison table (Infer vs. Semgrep)

